

How to write an amazing article

Adam Herout*

Abstract

What is the problem? What is the topic? The aim of this paper? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. How is the problem solved, the aim achieved (methodology)? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. Curabitur massa neque, commodo posuere fringilla ut, cursus at dui. Nulla quis purus a justo pellentesque. What are the specific results? How well is the problem solved? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. So what? How useful is this to Science and to the reader? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod.

Keywords: Keyword1 — Keyword2 — Keyword3

Supplementary Material: [Demonstation video](#) — [Downloadable code](#)

*herout@fit.vut.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

With the rise of Large Language Models (LLMs) and the increase in their capabilities, companies are motivated to use LLMs for automation tasks, including testing and the tooling for it. However, LLMs are not plug-and-play; specialized tooling is often required to produce reliable outputs. This work defines a formal base into which LLMs rewrite requirements and test cases written in natural language, so that the result is parsable by test coverage checking systems.

There is an increasing demand to instrument systems in natural language, including the specification of requirements and their test cases for testing systems and system components. Even when requirements and test cases are written in English, they must still be rewritten into a mathematical formal base so that existing test-coverage techniques can be applied. A formal base into which LLMs can convert human-written text is therefore needed. This formal base must be able to express all motivational requirement examples. Its semantics must be easily interpretable by LLMs and

agentic systems, so that agents can rewrite requirements and test case descriptions from human language into the formal base with high accuracy and without changing the semantics; the formalism itself must be unambiguous. It must also be convertible to existing formal bases used by formal verification tools. A good formalism should meet all of the mentioned criteria.

2. Existing methods of requirement formalisation – State of the Art

The translation from natural-language (NL) requirements to formal specifications has long been pursued through controlled-language frontends that constrain the input enough to make it unambiguous and further processable. The recent maturation of large language models has reopened the question of to what degree this translation can be automated without such restrictions, leading to LLM-based pipelines and hybrid approaches that pair LLMs with formal provers and solvers — the rest of this section surveys that landscape. This overview is informed by the recent short

survey of Beg et al. (2025) [1], which classifies the LLM-based literature in detail. It is further complemented with controlled-language frontends that the survey only briefly covers. Table 1 summarises the systems discussed in this section.

Controlled-language frontends. The longest-lived line of work attacks the problem at the *input* side by constraining the natural language itself. Nowakowski et al. (2013) [9] developed the *Requirements Specification Language* and the *ReDSeeDS* platform, which couples a constrained-NL requirements notation with automated transformations into code. Ghosh et al. (2016) [5] present *ARSENAL*, an NLP-based framework that extracts formal requirement specifications from natural language with automatic verification. NASA’s *FRET*, introduced by Giannakopoulou et al. (2020) [6], lets engineers write requirements in *FRETISH*, a restricted English with unambiguous semantics that maps deterministically to temporal logic.

LLM-only pipelines. With the rise of large language models, a growing body of work delegates the full translation to the LLM itself. Nayak et al. (2022) [8] present *Req2Spec*, an NLP pipeline that maps natural-language requirements onto the pattern-based formal specifications consumed by HANFOR; on 222 BOSCH automotive requirements it correctly formalises 71%. Cosler et al. (2023) [2] propose *nl2spec*, which uses few-shot prompting of Codex (code-davinci-002) and Bloom-176B to translate unstructured natural language into Linear-time Temporal Logic (LTL), with iterative user refinement of subformulas. Fang et al. (2024) [3] introduce *AssertLLM*, a three-stage pipeline of customised GPT-4o models (the final stage retrieval-augmented) that generates SystemVerilog Assertions from hardware design specifications with 89% correctness.

Hybrid LLM + prover systems. The most recent strand pairs an LLM with a formal prover or solver that has the final say on correctness. Jiang et al. (2022) [7] introduce *Thor*, which couples a 700M-parameter decoder-only transformer with the Isabelle/HOL theorem prover via Sledgehammer-based premise selection (“Hammers”), raising proof success on PISA from 39% to 57%. Yang et al. (2023) [11] release *LeanDojo* and the retrieval-augmented prover *ReProver*, a ByT5 encoder-decoder paired with premise retrieval over the Lean mathematics library and benchmarked on 98 734 theorems. Quan et al. (2024) [10] propose *ExplanationRefiner*, a refinement loop between an LLM (GPT-4, GPT-3.5-turbo, Llama2-70b, Mixtral-8x7b, or Mistral-small) and the Isabelle/HOL prover that validates and

Table 1. State-of-the-art systems for natural-language to formal-specification translation discussed in this section, in chronological order within each category. Abbreviations: HITL = human-in-the-loop; NLP = natural language processing; MDD = model-driven development; SAL = Symbolic Analysis Laboratory; LTL = linear-time temporal logic; HANFOR = an industrial requirements-analysis tool that consumes pattern-based formal specifications; SVA = SystemVerilog Assertions; HOL = higher-order logic (as in the Isabelle/HOL theorem prover); SMT = satisfiability modulo theories.

Ref.	System	Year	Models	Target	HITL
<i>Controlled-language frontends</i>					
[9]	RSL/ReDSeeDS	2013	— (rule-based)	Code (MDD)	Yes
[5]	ARSENAL	2016	NLP pipeline	SAL/LTL	No
[6]	FRET	2020	— (rule-based)	LTL	Yes
<i>LLM-only pipelines</i>					
[8]	Req2Spec	2022	NLP pipeline	HANFOR	No
[2]	nl2spec	2023	Codex, Bloom-176B	LTL	Yes
[3]	AssertLLM	2024	GPT-4o	SVA	No
<i>Hybrid LLM + prover systems</i>					
[7]	Thor	2022	700M trans-former	Isabelle/HOL	No
[11]	LeanDojo	2023	ByT5 + retrieval	Lean	No
[10]	ExplanationRefiner	2024	GPT-4/3.5, Llama2-70b, Mixtral-8x7b, Mistral-small	Isabelle/HOL	No
[4]	SAT-LLM	2024	LLM + SMT solver	SMT	No

corrects natural-language explanations. Fazelnia et al. (2024) [4] build *SAT-LLM*, which combines an LLM with an SMT solver to detect conflicting requirements at F1 0.91.

[[Add the gap and what is our contribution, basically summarize the next section]]

3. Contributing by designing a “close-to-NL” formalism

While prior controlled-language frontends [6, 9, 5] and LLM-based translators [2] target a single fixed logic — typically LTL or a tool-specific pattern language — none of the systems surveyed in Section 2 position their formalism around predicate-level matchability between requirements and test cases for coverage-gap detection, nor as a logic-agnostic intermediate explicitly designed to be the translation target for LLMs. The formalism presented in this section is designed around precisely these two properties.

109 3.1 Formalism requirements and its design

110 The formalism must satisfy the properties listed below.
111 Each property is paired with a concrete use case ex-
112 pressed in abstract placeholders (signals A, B ; events
113 e_A, e_B ; etc.) and a brief note on the construct the
114 formalism provides to address it.

115 **[[Fix ordering]]**

116 **[[Include a requirement and test case example,**
117 **so the reader can have a better understanding be-**
118 **fore reading the rest]]**

119 **Meta-level properties. Logic-agnostic representa-**
120 **tion of system behaviour in time.** The formalism
121 shall not commit to a specific underlying logic, so
122 that a single requirement representation can be retar-
123 geted to different verification back-ends. *Solution:* A
124 requirement is represented as a composition of two
125 kinds of building blocks: (i) ordinary expressions and
126 predicates that, when evaluated against a trace (the
127 same kind of object a test case produces), yield con-
128 crete values or truth values at each point in time; and
129 (ii) temporal predicates that aggregate those point-in-
130 time predicates into a single truth value over the entire
131 trace. The semantics of each building block is defined
132 directly in set-theoretic terms over traces and time,
133 independent of any target back-end. Every temporal
134 predicate has a direct LTL (or metric-LTL) equivalent
135 and a corresponding timed-automaton construction.

136 **Mechanical coverage-gap detection.** Given a set
137 of formalised requirements and a set of formalised test
138 cases, the formalism shall enable a checker to deter-
139 mine which requirement conditions are not exercised
140 by any test case. *Solution:* Requirements and test cases
141 are formalised over the same entities and the same
142 predicates, so that the same sub-formulas appear on
143 both sides. A checker can then evaluate, for each test
144 case, which meaningful combinations of sub-formulas
145 inside a requirement's formula are driven to true and
146 which to false, and from that compute the fraction of
147 relevant combinations that the test suite exercises.

148 **Extensible domains.** Real-world requirements
149 draw on an open-ended range of kinds of things in a
150 system (signals, registers, channels, states, and so on)
151 and of events that can happen to them (a register being
152 written, a channel transmitting a value, a state being
153 entered), and it is unrealistic to fix a closed catalogue
154 up front that covers every domain the formalism may
155 ever be applied to. The formalism shall therefore keep
156 these catalogues extensible. *Example:* To support a
157 new entity kind C representing network packets, only
158 the relevant catalogue is widened; the rest of the for-
159 malism is untouched. *Solution:* The set of entity kinds,
160 the set of value types, the assignment of permitted

attributes to each entity kind, and the assignment of 161
permitted events (parameterised by the entity they hap- 162
pen to) to each entity kind are all parameters of the 163
formalism rather than fixed sets. Adding a new entry 164
to any of them requires only registering it in the rele- 165
vant catalogue; existing constructs continue to work 166
unchanged. 167

Time and values. Discrete and continuous time. 168
The formalism shall support both discrete-time and 169
continuous-time requirements within a single vocabu- 170
lary. *Example:* A discrete-time requirement says “at 171
every step, A is incremented by 3”; a continuous-time 172
requirement says “ e_B must occur within 2 seconds of 173
 e_A ”. *Solution:* The time set T has two flavours: $T_D =$ 174
 $\mathbb{N} \cup \{\infty\}$ for discrete time and $T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$ for 175
continuous time. Each requirement carries a *flavour* $\in$ 176
 $\{\text{Discrete, Continuous}\}$ field that fixes T for its scope. 177

Entities of multiple kinds carry values, and op- 178
erations modify them in test cases. The formalism 179
shall represent entities of different kinds (registers, 180
states, communication channels, computed signals, 181
etc.) and let each carry a value at any given time 182
point. It shall also provide a way for test cases to de- 183
scribe how those values change over time. *Example:* 184
Register A , state B , and channel C each have a value 185
at every relevant time point; a test case may, for in- 186
stance, write a new value into A , transition B to a new 187
state, or transmit a value through C . *Solution:* The 188
formalism defines an extensible catalogue of entity 189
kinds (currently signal, storage, event, event-trigger, 190
channel, state, value, abstract, and set), and a valua- 191
tion maps each entity to its value at each time point of 192
a trace. Modifications of those values are expressed 193
through *operations* — an extensible catalogue of ac- 194
tions, currently including *calculate*, *write*, *read*, *fire*, 195
set_state, *transmit*, *receive*, *insert*, *remove*, and *clear* 196
— that a test case fires at specific time points to drive 197
the trace. Each entity kind has its own subset of appli- 198
cable operations (e.g. *write* applies to storage, *fire* to 199
event-triggers, *transmit* to channels). 200

Initial-value constraints. The formalism shall be 201
able to constrain the value of an entity at the initial 202
time point. *Example:* “Signal A has initial value 0.” *So-* 203
lution: The temporal construct $\text{Initial}(\psi) \Leftrightarrow \psi(p, 0)$ 204
asserts a point-in-time predicate at time 0. 205

Reference to an entity's value at another time 206
point. The formalism shall allow a constraint to refer 207
to the value an entity had (or will have) at an ear- 208
lier or later time point, enabling recurrences, toggles, 209
and look-ahead conditions. *Example:* “ $A(\text{Now}) =$ 210
 $A(\text{Now} - 1) + 3$ ”; “ B equals the negation of B 's previ- 211
ous value”; or “ C at the next step equals the current 212

213 value of D ". *Solution*: $Val_p(e, t)$ returns the value of
 214 e at any time t , past or future; for discrete time, $Prev$,
 215 $Next$, and $RelTime$ shift the time argument backward
 216 or forward; $ValBefore_p(e, t)$ exposes the left limit at t .

217 **Arithmetic on values.** The formalism shall sup-
 218 port standard arithmetic on numeric values so that
 219 constraints can express computations over entity val-
 220 ues. *Example*: " $A = B + 3$ "; or " $C = 1.2 \cdot D$ ". *Solution*:
 221 The standard arithmetic operators $+$, $-$, $\times$, $/$ are de-
 222 fined on numeric values and can appear inside any
 223 expression.

224 **Time arithmetic and intervals.** The formalism
 225 shall support arithmetic on time points and the explicit
 226 construction of intervals with controlled endpoint open-
 227 ness. *Example*: "the duration elapsed between e_A and
 228 e_B "; "the half-open interval $(t_1, t_2]$ ". *Solution*: $Diff$
 229 returns the duration between two time points; the four
 230 interval constructors $MkIntervalCC$, $MkIntervalOC$,
 231 $MkIntervalCO$, $MkIntervalOO$ build intervals with the
 232 chosen open/closed endpoints.

233 **Static entity-attribute access.** The formalism
 234 shall expose static attributes of an entity (independent
 235 of trace and time). *Example*: The address of register
 236 A is the constant $0xAA000018$ and can be referenced
 237 as such. *Solution*: Static attributes are stored in an en-
 238 tity's modifier map $\mu_e : ModKey \rightarrow ModVal$; accessors
 239 such as $Addr(e) = \mu_e(address)$ retrieve them without
 240 reference to p or t .

241 **Events and triggers. Event occurrence at a given**
 242 **time point.** The formalism shall express that an event
 243 occurs at a particular time point. *Example*: " e_A is hap-
 244 pening at t ". *Solution*: The predicate $Happening(e, t)$,
 245 where e is an entity of type `Event`.

246 **Past-event semantics.** The formalism shall ex-
 247 press that an event has occurred at some point up to
 248 the current time. *Example*: " e_A has happened at least
 249 once." *Solution*: The predicate $HasHappened_p(e, t)$.

250 **Reference to event-occurrence times.** The for-
 251 malism shall expose the time of a specific past or future
 252 occurrence of an event. *Example*: "the time of the most
 253 recent occurrence of e_A ". *Solution*: $LastOcc_p(ev, t, n)$
 254 returns the interval spanning the n most recent occur-
 255 rences of ev up to t ; $FirstOcc_p$ does the same in the
 256 forward direction.

257 **Counting event occurrences within an interval.**
 258 The formalism shall count how many times an event
 259 occurred within a given interval, with control over
 260 whether interval endpoints are included. *Example*:
 261 "the number of occurrences of e_A in the half-open in-
 262 terval $(t_1, t_2]$ ". *Solution*: $EvtOccCount_p(ev, i)$ returns
 263 the count.

Conditional and logical structure. Value-conditional
branches. The formalism shall support constraints
 that branch on the current value of an entity. *Exam-*
ple: "If A is true then B equals X ; otherwise B equals
 Y ." *Solution*: A value-guarded branch is expressed
 as the conjunction of two implications using $Allof$,
 $Implies$, and the comparison predicate Cmp , for ex-
 ample: $Allof(\{Implies(Cmp(A, =, true), Cmp(B, =$
 $, X)), Implies(Cmp(A, =, false), Cmp(B, =, Y))\})$.

Conjunction, disjunction, and negation of pred-
icates. The formalism shall combine predicates of
 any kind (event happenings, value comparisons, oc-
 currence counts, historical values) into a single con-
 junctive or disjunctive guard, and shall negate any
 predicate. *Example*: " e_A and $B > 100$ and e_C has oc-
 curred at least twice"; " e_A or e_B or e_C "; or " A is not
 equal to B ". *Solution*: $Allof$, $AnyOf$, and Not form the
 conjunction, disjunction, and negation of predicates
 drawn from the shared predicate space.

Sets, set membership, and set quantification.
 The formalism shall represent finite sets of values, test
 whether a value belongs to a set, quantify over a finite
 set of entities, and reason about properties of the subset
 that satisfies a predicate. *Example*: "the value v be-
 longs to set S "; "for every entity in $\{A, B, C, D\}$, predi-
 cate P holds"; or "at most two entities in $\{A, B, C, D\}$
 have a value greater than 10." *Solution*: The Set entity
 kind carries set values; $Contains$ tests membership; the
 quantifiers $ForAll(S, P)$ and $Exists(S, P)$ range over a
 finite set, with $Filter$ and $Size$ supporting cardinality
 reasoning over the subset satisfying a predicate.

Temporal patterns. Sequencing and time-bounded
causation. The formalism shall require multiple ac-
 tions to occur in a specified order, and shall require an
 effect to occur within a bounded duration of a cause.
Example: " e_A then e_B then e_C "; "if e_A occurs, e_B must
 occur within 2 seconds". *Solution*: $Sequence(\psi_1, \dots, \psi_n)$
 and $CausesWithin(\psi_{cond}, \psi_{effect}, d)$, respectively; sim-
 ilarly, $Causes(\psi_{cond}, \psi_{effect})$ for unbounded causation,
 $Precedes(\psi_1, \psi_2)$ for ordering, $Excludes(\psi_1, \psi_2)$ for
 mutual exclusion, and $Immediately(\psi_1, \psi_2)$ for next-
 step succession.

Existence, global constraints, and trace-level
composition. The formalism shall express both "at
 some point in the trace" and "at every time point in
 the trace", and shall let multiple such trace-level con-
 straints be combined into a single requirement. *Ex-*
ample: "Eventually A equals 5"; " A is always non-
 negative"; or "Always ψ_1 and eventually ψ_2 ". *So-*
lution: $Eventually(\psi)$ and $Always(\psi)$ produce trace-
 level truth values; multiple trace predicates compose
 via the standard logical operators ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$)

316 applied at the trace level.

317 **Sliding temporal windows.** The formalism shall
318 reason about a window of fixed length ending at the
319 current time. *Example:* “ e_A occurred at least twice
320 in the last 10 seconds.” *Solution:* An interval con-
321 structor applied to *Now* (e.g. $MkIntervalOO(Now -$
322 $d, Now)$) produces the window, which can then be
323 passed to $EvtOccCount_p$ to count event occurrences, to
324 $MaxVal_p/MinVal_p$ to bound a signal’s range, or to inter-
325 val predicates such as $IntervalIncludes/IntervalExcludes$.
326 **[[Preferably without human in the loop]]**

327 3.2 Formalism design decisions

328 **[[Might already be covered by previous section]]**

329 3.3 Formalism definition

330 **[[Add an appendix]] [[Add appendix with exam-**
331 **ples]]**

332 3.4 Known limitations and possible future ex-

333 **tensions**

334 **[[Although this work claims that the formalism**
335 **should be close to english and is close in the sense**
336 **that it uses english words, the words themselves**
337 **might not appear in a similar order than they would**
338 **in english. This might be a limitation.]]**
339 **[[Add limitations from formalism document]]**

340 3.5 Compatibility with well-known formal spec-

341 **ification methods**

342 3.6 Requirement and test case examples

343 3.7 Future work

344 4. Conclusions

345 Acknowledgements

346 I would like to thank my supervisor Ing. Aleš Smrčka
347 Ph.D. for his help.

348 References

- 349 [1] BEG, A., O’DONOGHUE, D. and MONAHAN, R.
350 *A Short Survey on Formalising Software Require-*
351 *ments with Large Language Models.* 2025. Avail-
352 able at: [https://arxiv.org/abs/2506.](https://arxiv.org/abs/2506.11874)
353 [11874](https://arxiv.org/abs/2506.11874).
- 354 [2] COSLER, M., HAHN, C., MENDOZA, D.,
355 SCHMITT, F. and TRIPPEL, C. *NI2spec: Inter-*
356 *actively Translating Unstructured Natural Lan-*
357 *guage to Temporal Logics with Large Language*
358 *Models.* 2023. Available at: [https://arxiv.](https://arxiv.org/abs/2303.04864)
359 [org/abs/2303.04864](https://arxiv.org/abs/2303.04864).

- [3] FANG, W., LI, M., LI, M., YAN, Z., LIU, S. 360
et al. AssertLLM: Generating Hardware Verifi- 361
cation Assertions from Design Specifications via 362
Multi-LLMs. In: *2024 IEEE LLM Aided Design* 363
Workshop (LAD). 2024, p. 1–1. 364
- [4] FAZELNIA, M., MIRAKHORLI, M. 365
and BAGHERI, H. Translation Titans, Reasoning 366
Challenges: Satisfiability-Aided Language 367
Models for Detecting Conflicting Requirements. 368
In: *Proceedings of the 39th IEEE/ACM Inter-* 369
national Conference on Automated Software 370
Engineering (ASE ’24). New York, NY, USA: 371
Association for Computing Machinery, 2024, 372
p. 2294–2298. 373
- [5] GHOSH, S., ELENIOUS, D., LI, W., LINCOLN, 374
P., SHANKAR, N. et al. ARSENAL: Auto- 375
matic Requirements Specification Extraction 376
from Natural Language. In: RAYADURGAM, S. 377
and TKACHUK, O., ed. *NASA Formal Methods.* 378
Cham: Springer International Publishing, 2016, 379
p. 41–46. 380
- [6] GIANNAKOPOULOU, D., MAVRIDOU, A., 381
RHEIN, J., PRESSBURGER, T., SCHUMANN, J. 382
et al. Formal Requirements Elicitation with 383
FRET. In: *Joint Proceedings of REFSQ-2020* 384
Workshops, Doctoral Symposium, Live Studies 385
Track, and Poster Track. Pisa, Italy: [b.n.], 2020. 386
- [7] JIANG, A. Q., LI, W., TWORKOWSKI, S., 387
CZECHOWSKI, K., ODRZYGÓDŹ, T. et al. Thor: 388
Wielding Hammers to Integrate Language Mod- 389
els and Automated Theorem Provers. In: *Ad-* 390
vances in Neural Information Processing Sys- 391
tems. Curran Associates, Inc., 2022, p. 8360– 392
8373. 393
- [8] NAYAK, A., TIMMAPATHINI, H. P., MURALI, 394
V., PONNALAGU, K., VENKOPARAO, V. G. 395
et al. Req2Spec: Transforming Software Re- 396
quirements into Formal Specifications Using Nat- 397
ural Language Processing. In: *Requirements* 398
Engineering: Foundation for Software Quality 399
(REFSQ 2022). Berlin, Heidelberg: Springer- 400
Verlag, 2022, p. 87–95. 401
- [9] NOWAKOWSKI, W., ŚMIAŁEK, M., AM- 402
BROZIEWICZ, A. and STRASZAK, T. 403
Requirements-Level Language and Tools 404
for Capturing Software System Essence. *Com-* 405
puter Science and Information Systems. 2013, 406
vol. 10, no. 4, p. 1499–1524. 407
- [10] QUAN, X., VALENTINO, M., DENNIS, L. A. 408
and FREITAS, A. *Verification and Refinement* 409

of Natural Language Explanations through LLM-Symbolic Theorem Proving. 2024. Available at: <https://arxiv.org/abs/2405.01379>.

[11] YANG, K., SWOPE, A., GU, A., CHALAMALA, R., SONG, P. et al. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023, p. 21573–21612.